SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Name of the Student	**K.Neeha**
Reg. No	**192425245**
GitHub Link	**https://github.com/192425245simats-oss/192425245-MLA0403.git**

EXPERIMENTS

COURSE NAME: Deep Learning **COURSE CODE: MLA0403**
FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Name of the student:**

QUESTIONS:10 **Total Marks:50**

CO1- AT3 Experiments
Questions

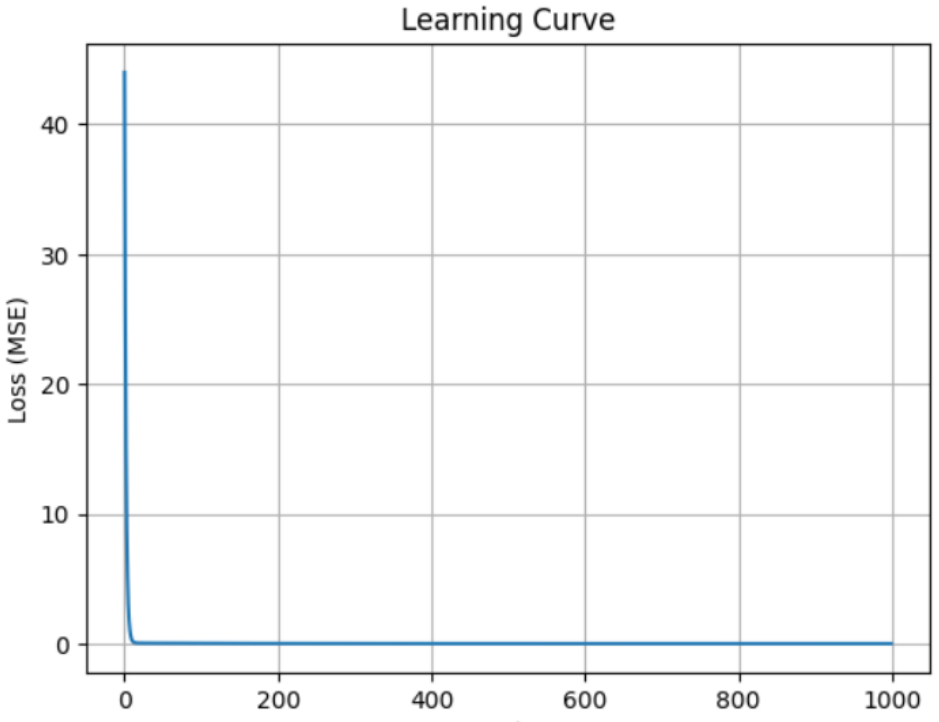
1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Answer :

```
plt.show()
```

```
Weight = 1.995
Bias = 0.017
Final Loss = 5.5e-05
```



2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer :

```
data = np.random.normal(true_mean, true_std, 1000)

# MLE estimates
estimated_mean = np.mean(data)
estimated_variance = np.mean((data - estimated_mean) ** 2)

print("Actual Mean:", true_mean)
print("Estimated Mean:", round(estimated_mean, 3))

print("Actual Variance:", true_std**2)
print("Estimated Variance:", round(estimated_variance, 3))
```

```
Actual Mean: 10
Estimated Mean: 10.039
Actual Variance: 4
Estimated Variance: 3.832
```

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Answer :

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train model
model = LogisticRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluation
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("Accuracy:", round(accuracy * 100, 2), "%")
print("\nConfusion Matrix:")
print(cm)
```

```
Accuracy: 100.0 %

Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer :

```
                    max_iter=1000,
                    random_state=42)

# Train model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", round(accuracy * 100, 2), "%")
```

```
Accuracy: 96.67 %
/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: Convergenc
  warnings.warn(
```

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer :

```
5]
0s
```

```python
import numpy as np

# Activation Functions
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def relu(x):
    return np.maximum(0, x)

# Input values
inputs = np.array([-2, -1, 0, 1, 2])

# Outputs
sigmoid_output = sigmoid(inputs)
relu_output = relu(inputs)

print("Inputs:", inputs)
print("Sigmoid Output:", np.round(sigmoid_output, 3))
print("ReLU Output:", relu_output)
```

```
Inputs: [-2 -1  0  1  2]
Sigmoid Output: [0.119 0.269 0.5   0.731 0.881]
ReLU Output: [0 0 0 1 2]
```

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Answer :

```
X_train, X_test, y_train, y_test = train_test_split(
    X1, y1, test_size=0.2, random_state=42)

model = Perceptron(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Linear Dataset Accuracy:", accuracy_score(y_test, y_pred))

# Non-Linearly Separable Dataset
X2, y2 = make_circles(
    n_samples=200,
    noise=0.1,
    factor=0.5,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X2, y2, test_size=0.2, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Non-Linear Dataset Accuracy:", accuracy_score(y_test, y_pred))
```

```
Linear Dataset Accuracy: 0.85
Non-Linear Dataset Accuracy: 0.55
```

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Answer :

7. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

[8]
0s

```python
import numpy as np

# Inputs
x1 = 1
x2 = 2

# Weights
w1 = 0.5
w2 = 0.4

# Bias
b = 0.1

# Sigmoid function
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# Forward Propagation
z = x1 * w1 + x2 * w2 + b
output = sigmoid(z)

print("Weighted Sum (z):", round(z, 2))
print("Neuron Output:", round(output, 4))
```

```
Weighted Sum (z): 1.4
Neuron Output: 0.8022
```

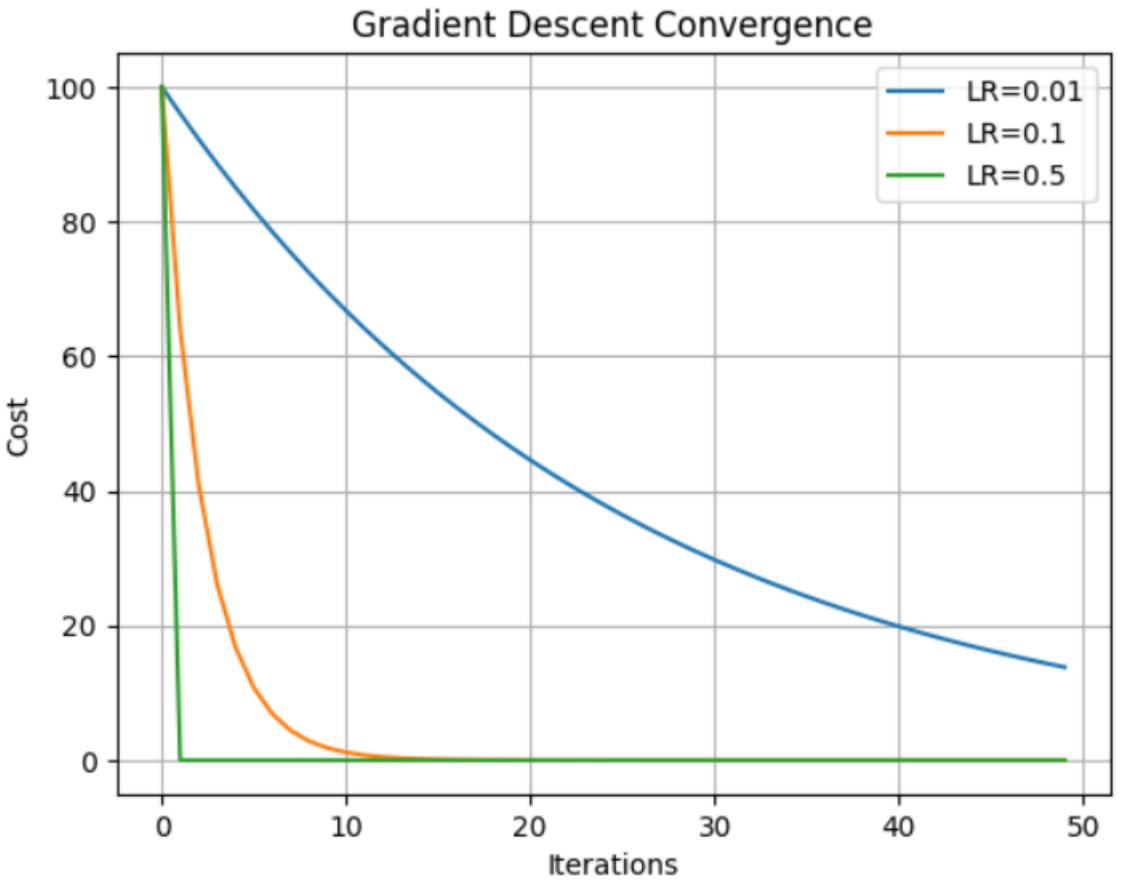
:

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a)Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer :

```
plt.xlabel("Iterations")
plt.ylabel("Cost")
plt.legend()
plt.grid(True)
plt.show()
```



b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer :

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

[11] ✓ 12s

```
X, y = make_regression(n_samples=500, n_features=1,
                       noise=10, random_state=42)

batch_sizes = [8, 32, 128]

for batch in batch_sizes:
    model = SGDRegressor(max_iter=1,
                         learning_rate='constant',
                         eta0=0.01,
                         random_state=42)

    for epoch in range(100):
        indices = np.random.permutation(len(X))
        X_shuffled = X[indices]
        y_shuffled = y[indices]

        for i in range(0, len(X), batch):
            X_batch = X_shuffled[i:i+batch]
            y_batch = y_shuffled[i:i+batch]
            model.partial_fit(X_batch, y_batch)

    predictions = model.predict(X)
    mse = mean_squared_error(y, predictions)
    print("Batch Size:", batch, "MSE:", round(mse, 2))
```

```
Batch Size: 8 MSE: 99.9
Batch Size: 32 MSE: 98.74
Batch Size: 128 MSE: 99.69
```

Answer :

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer :

```
    model = KNeighborsClassifier(n_neighbors=3)
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)

    print(f"Dimensions: {d}")
    print(f"Average Distance: {avg_distance:.2f}")
    print(f"Accuracy: {accuracy:.2f}")
    print()
```

```
Dimensions: 2
Average Distance: 2.38
Accuracy: 0.93

Dimensions: 10
Average Distance: 4.70
Accuracy: 0.87

Dimensions: 50
Average Distance: 10.10
Accuracy: 0.66

Dimensions: 100
Average Distance: 14.23
Accuracy: 0.71
```